



DIPARTIMENTO DI ELETTRONICA, INFORMAZIONE E BIOINGEGNERIA



**POLITECNICO
DI MILANO**

Fondamenti di informatica



Introduzione al linguaggio C



- Linguaggio di programmazione ad alto livello
- Sviluppato negli anni 70 (C – standard ANSI)
- Molto diffuso e adatto ad un ampio spettro di applicazioni:
 - Scientifiche
 - Gestionali
 - Industriali, es. acquisizione dei dati, controllo dei processi
 - Informatiche, es. SW di base (Sistema operativo Unix), strumenti (CAD), pacchetti applicativi
 - Sistemi dedicati, es. microcontrollori, computer di bordo, domotica
- **Noi studieremo la versione C89**



- È un linguaggio imperativo
 - il programma è composto come un insieme di istruzioni (dette anche direttive o comandi), ciascuna delle quali può essere pensata come un “ordine” che viene impartito al calcolatore
- È un linguaggio dichiarativo
 - L'intenzione di utilizzare una variabile o una funzione deve essere dichiarata a priori
- È un linguaggio strutturato
 - Fornisce i costrutti base per la definizione di sequenza, selezione ed iterazione
- È un linguaggio case sensitive
 - Discrimina tra caratteri maiuscoli e minuscole
 - e.g., `Main` è diverso da `main`!



- Un programma scritto in C è un file di testo strutturato in modo opportuno
- Il programma scritto in modo corretto può essere compilato, cioè tradotto in linguaggio macchina
- Imparare a programmare in C significa impararne la sintassi e la semantica per fare in modo che esso:
 - Sia compilabile (verifica sintattica)
 - Si comporti come atteso (verifica semantica)



- Un esempio molto semplice di programma:

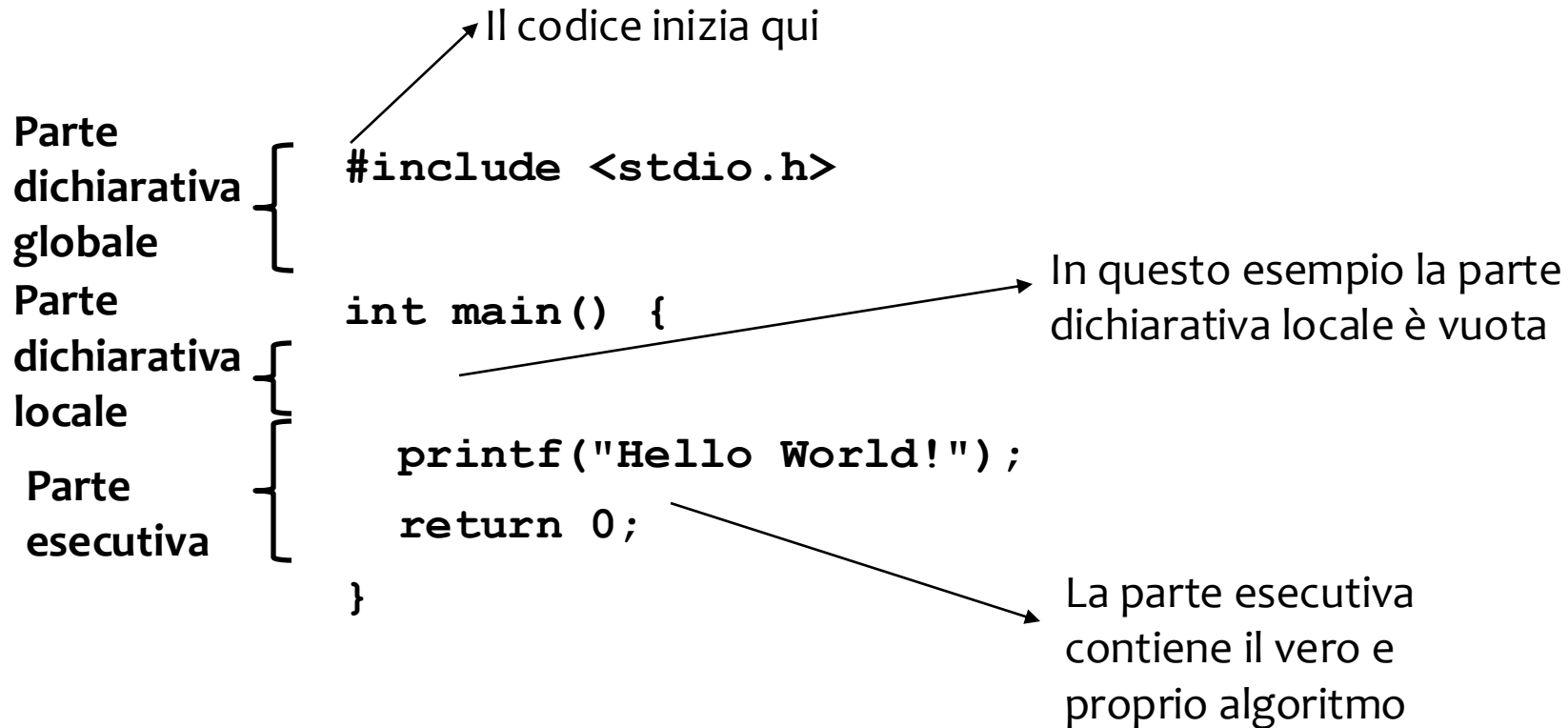
```
#include <stdio.h>

int main() {

    printf("Hello World!");
    return 0;
}
```

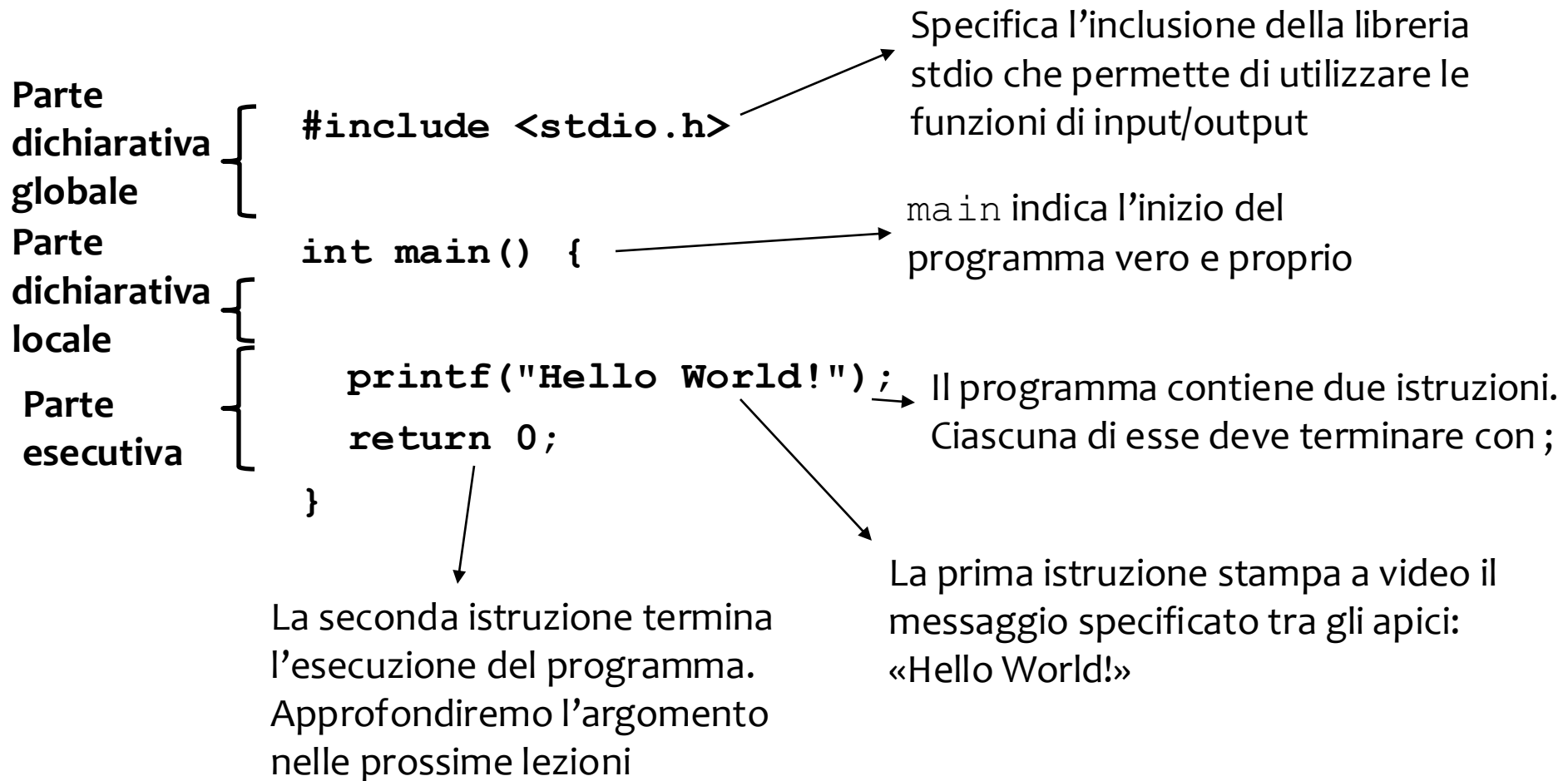


- Un esempio molto semplice di programma:





- Un esempio molto semplice di programma:





- Un esempio molto semplice di programma:

```
/* Il mio primo programma */
```

Parte
dichiarativa
globale

```
#include <stdio.h>
```

Parte
dichiarativa
locale

```
int main() {
```

Parte
esecutiva

```
    printf("Hello World!");  
    return 0;
```

```
}
```

Possiamo aggiungere commenti dove si vuole (testo tra `/* */`). Servono ai programmatori per rendere il codice più leggibile; sono ignorati dal compilatore

Possiamo mettere quanti a-capo e spazi vogliamo perché vengono ignorati.
In realtà utilizzeremo una convenzione detta “indentazione” per rendere il codice leggibile



E ora che abbiamo il codice?



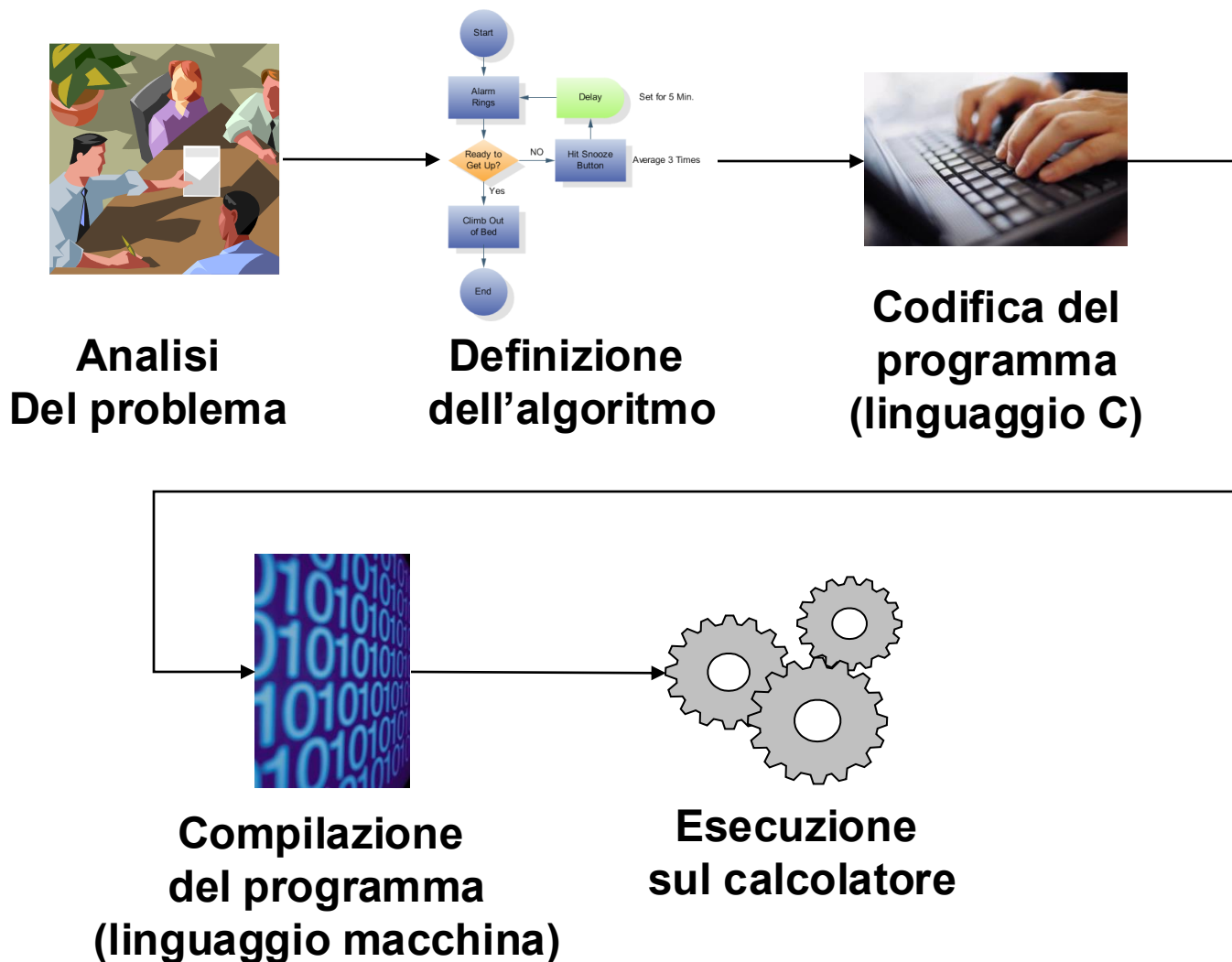
- Dobbiamo trovare un elaboratore in grado di eseguirlo
- Problema: il calcolatore è in grado di elaborare istruzioni in linguaggio macchina
- Dobbiamo quindi trovare anche un modo per trasformare il C (linguaggio di alto livello) in linguaggio macchina (linguaggio di basso livello)



- Un elaboratore è un insieme di circuiti
- Si basa su logica binaria
- Il linguaggio C è vicino al modo di pensare umano
 - È infatti un linguaggio di alto livello
- Il linguaggio macchina è vicino al modo di lavorare di un elaboratore
 - È infatti un linguaggio di basso livello



Il flusso di sviluppo di un programma





- Un esempio molto semplice di programma:

```
/* Il mio primo programma */  
#include <stdio.h>  
  
int main() {  
    printf("Hello World!");  
    return 0;  
}
```

- L'esecuzione del programma porta al seguente output sul terminale:

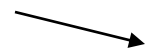
```
Hello World!
```



- Potreste imbattervi anche in programmi che hanno un `main` leggermente diverso

```
#include <stdio.h>
```

Parola chiave
`void` al posto
di `int`



```
void main() {
```

```
    printf("Hello World!");
```

Nessuna istruzione
`return` alla fine
del `main`

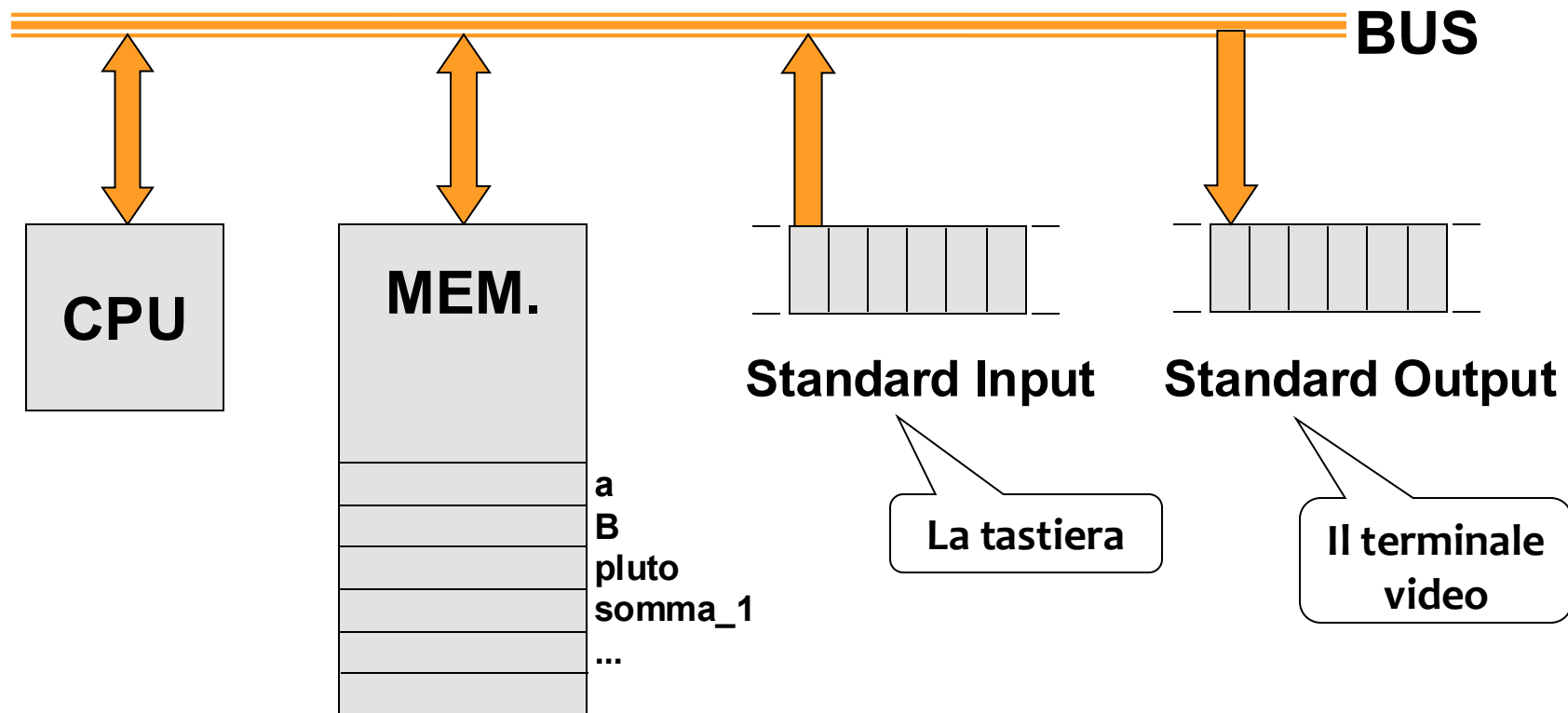


```
}
```

- Discuteremo questi dettagli nelle prossime lezioni

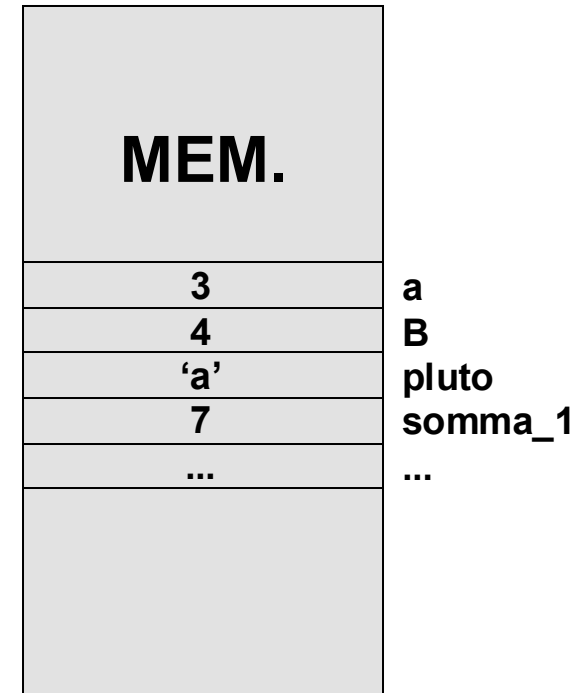


- Quando si scrive un programma in C si immagina di avere una macchina un po' più astratta rispetto al calcolatore reale





- Divisa in celle elementari
- Ogni cella può contenere un dato
- I dati possono essere
 - Numeri
 - Caratteri
 - Stringhe (sequenze di caratteri in celle adiacenti)
 - ...
- Semplificazioni / idealizzazioni / approssimazioni / astrazioni
 - Nessun limite al numero delle celle
 - Nessun limite ai valori numerici contenuti





- La macchina astratta del C ha due periferiche “standard” di ingresso e uscita
 - `stdin`: standard input che acquisisce i dati da tastiera
 - `stdout`: standard output che visualizza i messaggi nel terminale video
- che possono essere viste come “sequenze” (flussi) di byte, normalmente interpretati come caratteri



- L'unità centrale di elaborazione (CPU) è in grado di eseguire una alla volta in sequenza le istruzioni specificate nel programma
- Ciascuna istruzione può contenere:
 - Espressioni aritmetico/logica/relazionali aventi come operandi variabili contenute nella memoria centrale e salvando ciascun risultato in una variabile in memoria
 - Letture dallo standard input che salvano i dati acquisiti in variabili contenute in memoria
 - Scritture sullo standard output di messaggi contenenti anche i dati salvati nelle variabili in memoria
 - Istruzioni di controllo che modificano il flusso sequenziale di esecuzione



- Nella parte dichiarativa globale vengono specificati (nell'ordine)
 - Le direttive `include` con cui si specifica quali librerie vengono usate
 - Le direttive `define` per definire costanti da utilizzare nel programma
 - La definizione di nuovi tipi di dato (vedremo più avanti)
 - La dichiarazione di variabili globali (non le useremo in questo corso)
- Nella parte dichiarativa locale vengono dichiarate le variabili
- Nella parte esecutiva vengono specificate le istruzioni che implementano l'algoritmo



**Parte
dichiarativa
globale** { `#include <stdio.h>`
`#define PI 3.14`

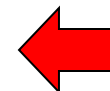
**Parte
dichiarativa
locale** { `int main() {`
`float r, a;`
**Parte
esecutiva** { `scanf("%f", &r);`
`a = r * r * PI;`
`printf("%f", a);`
`return 0;`
`}`

(il programma chiede all'utente la lunghezza del raggio di un cerchio, ne calcola l'area e visualizza il risultato)



Parte
dichiarativa
globale

```
#include <stdio.h>  
#define PI 3.14
```



Direttive include

Parte
dichiarativa
locale

Parte
esecutiva

```
int main() {  
    float r, a;  
    scanf("%f", &r);  
    a = r * r * PI;  
    printf("%f", a);  
    return 0;  
}
```



- Le direttive:
 - sono istruzioni particolari rivolte al compilatore
 - iniziano con il carattere `#` e non terminano con `;`
- La direttiva `include` indica l'intenzione di utilizzo di una libreria esterna (cioè una collezione di funzioni, cioè sottoprogrammi che svolgono un compito più complesso di una singola istruzione)
- Esempio
 - `#include <stdio.h>` richiede la libreria di gestione degli input e degli output (`printf()`, `scanf()`, ...)
 - `#include <math.h>` richiede la libreria con le principali funzioni matematiche (`abs()`, `sin()`, ...)
- Una volta inclusa una libreria è possibile utilizzare tutte le funzioni definite al loro interno



```
Parte  
dichiarativa { #include <stdio.h>  
globale       { #define PI 3.14  
  
Parte  
dichiarativa { int main() { ← main  
locale       { float r, a;  
               scanf("%f", &r);  
Parte  
esecutiva    { a = r * r * PI;  
               printf("%f", a);  
               return 0;  
               }
```



- È una parola chiave e indica l'inizio del programma vero e proprio
- Le istruzioni che seguono saranno quelle effettivamente eseguite dal calcolatore
- Le istruzioni che compongono il programma principale (cioè il `main`) sono racchiuse all'interno di parentesi graffe { }



Parte
dichiarativa
globale { `#include <stdio.h>`
`#define PI 3.14`

Parte
dichiarativa
locale { `int main() {`
`float r, a;` ← Dichiarazioni variabili
`scanf("%f", &r);`
`a = r * r * PI;`
Parte
esecutiva { `printf("%f", a);`
`return 0;`
`}`



- Sintassi C per la dichiarazione di variabili

```
tipo_variabile nome_var;
```

- Possiamo anche dichiarare più variabili dello stesso tipo separandole con una virgola

```
tipo_variabile nome_var1, nome_var2;
```

- Esempi:
 - `int raggio, circonferenza;`
 - `char carattere;`
- Le variabili vanno dichiarate prima di qualsiasi istruzione eseguibile
- Il nome di una variabile deve essere **univoco** (cioè non deve essere già stato utilizzato) e **non deve essere una parola riservata**



- La dichiarazione di variabile indica che e come una variabile verrà utilizzata dal programma
 - Definisce l'identificatore simbolico (nome) **univoco**
 - Definisce il tipo, adatto ai valori da rappresentare (intero, carattere, ...)
 - Riserva (alloca) una locazione di memoria di dimensione adeguata a contenere il tipo
 - Associa in modo univoco l'indirizzo di memoria al nome
 - Consente di rilevare, durante la compilazione, errori sull'uso improprio della variabile nel programma



- Una variabile identifica uno spazio della memoria centrale
- Prima dell'esecuzione del programma deve essere chiaro quanto spazio serve al programma
 - Quindi devono essere note tutte le variabili che un programma intende utilizzare
- Tutti gli accessi alla variabile indicano un accesso alla memoria
 - Quindi deve essere chiaro quale spazio di memoria è associato alla variabile
- Durante l'esecuzione del programma le variabili hanno sempre un valore ben definito
 - Può essere significativo o non significativo, ma c'è



- Un tipo identifica la classe dei valori ammissibili, le operazioni lecite, e la modalità con cui agiscono le operazioni
- Esistono due macro-classi di tipi:
 - Tipi built-in (predefiniti)
 - Tipi definiti dall'utente (prossime lezioni...)
- Sia quelli built-in che quelli definiti dall'utente possono essere:
 - Semplici
 - Composti (prossime lezioni...)



`char`

- Rappresenta i caratteri in codifica ASCII (American Standard Code for Information Interchange)
- Occupa 8 bit (1 byte)
- I caratteri sono rappresentati in memoria tramite un valore numerico nell'intervallo da 0 a 255
- I valori di tipo `char` sono rappresentati nel codice di un programma tra apici semplici: `'1'`, `'a'`
- Caratteri speciali sono rappresentati mediante una specifica sequenza di due caratteri che inizia con `\`
 - `'\n'`: a capo, `'\t'`: tab

- Esempio:

```
char carattere1;  
char carattere2, carattere3;
```



TABELLA ASCII

Byte	Cod.	Char	Byte	Cod.	Char	Byte	Cod.	Char	Byte	Cod.	Char
00000000	0	Null	00100000	32	Spc	01000000	64	@	01100000	96	`
00000001	1	Start of heading	00100001	33	!	01000001	65	A	01100001	97	a
00000010	2	Start of text	00100010	34	"	01000010	66	B	01100010	98	b
00000011	3	End of text	00100011	35	#	01000011	67	C	01100011	99	c
00000100	4	End of transmit	00100100	36	\$	01000100	68	D	01100100	100	d
00000101	5	Enquiry	00100101	37	%	01000101	69	E	01100101	101	e
00000110	6	Acknowledge	00100110	38	&	01000110	70	F	01100110	102	f
00000111	7	Audible bell	00100111	39	'	01000111	71	G	01100111	103	g
00001000	8	Backspace	00101000	40	(	01001000	72	H	01101000	104	h
00001001	9	Horizontal tab	00101001	41	)	01001001	73	I	01101001	105	i
00001010	10	Line feed	00101010	42	*	01001010	74	J	01101010	106	j
00001011	11	Vertical tab	00101011	43	+	01001011	75	K	01101011	107	k
00001100	12	Form Feed	00101100	44	,	01001100	76	L	01101100	108	l
00001101	13	Carriage return	00101101	45	-	01001101	77	M	01101101	109	m
00001110	14	Shift out	00101110	46	.	01001110	78	N	01101110	110	n
00001111	15	Shift in	00101111	47	/	01001111	79	O	01101111	111	o
00010000	16	Data link escape	00110000	48	0	01010000	80	P	01110000	112	p
00010001	17	Device control 1	00110001	49	1	01010001	81	Q	01110001	113	q
00010010	18	Device control 2	00110010	50	2	01010010	82	R	01110010	114	r
00010011	19	Device control 3	00110011	51	3	01010011	83	S	01110011	115	s
00010100	20	Device control 4	00110100	52	4	01010100	84	T	01110100	116	t
00010101	21	Neg. acknowledge	00110101	53	5	01010101	85	U	01110101	117	u
00010110	22	Synchronous idle	00110110	54	6	01010110	86	V	01110110	118	v
00010111	23	End trans. block	00110111	55	7	01010111	87	W	01110111	119	w
00011000	24	Cancel	00111000	56	8	01011000	88	X	01111000	120	x
00011001	25	End of medium	00111001	57	9	01011001	89	Y	01111001	121	y
00011010	26	Substitution	00111010	58	:	01011010	90	Z	01111010	122	z
00011011	27	Escape	00111011	59	;	01011011	91	[	01111011	123	{
00011100	28	File separator	00111100	60	<	01011100	92	\	01111100	124	
00011101	29	Group separator	00111101	61	=	01011101	93	]	01111101	125	}
00011110	30	Record Separator	00111110	62	>	01011110	94	^	01111110	126	~
00011111	31	Unit separator	00111111	63	?	01011111	95	_	01111111	127	Del



`int`

- Rappresenta i numeri interi con segno
- I valori sono codificati internamente in complemento a due
- L'occupazione dipende dal tipo di architettura del calcolatore
- In genere nei calcolatori moderni occupa 32 bit (4 byte) e rappresenta i valori da -2147483648 a $+2147483647$

- Esempio:

```
int n01;
```

```
int numero5, num5;
```



`float`

- Rappresentano i razionali in virgola mobile
- Valori codificati mediante lo standard IEEE754 singola precisione
- Occupa 32 bit (4 byte)
- Valori da $-10E-38$ a $10E+38$

- Esempio:

```
float num7;
```




double

- Rappresentano i razionali in virgola mobile
- Valori codificati mediante lo standard IEEE754 doppia precisione
- Occupa 64 bit (8 byte)
- Valori da $-10E-308$ a $10E+308$
- Esempio:
`double base, altezza;`



Parte
dichiarativa
globale { `#include <stdio.h>`
 `#define PI 3.14` ← Dichiarazioni costanti

Parte
dichiarativa
locale { `int main() {`
 `float r, a;`
 `scanf("%f", &r);`
Parte
esecutiva { `a = r * r * PI;`
 `printf("%f", a);`
 `return 0;`
 `}`



- Esistono due tipi di costanti
 - Esplicite: esprimono direttamente dei valori
 - 24 costante di tipo `int`
 - 3.145 costante di tipo `double`
 - 'A' costante di tipo `char`
 - Simboliche
 - Sono nomi simbolici che il programmatore adotta per indicare dei valori prefissati (maggiore leggibilità)
 - Hanno un tipo espresso implicitamente nel valore
 - Devono essere precedentemente dichiarate



- Dichiarare una costante significa associare un simbolo ad un valore
- A differenza delle variabili tale valore non cambierà mai durante tutto il programma
- Per dichiarare una costante utilizziamo la direttiva `#define`



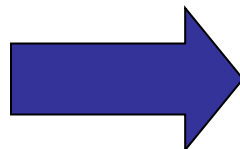
- La direttiva associa un nome ad un valore di qualunque tipo
- Ogni direttiva (specificata nella parte delle dichiarazioni) definisce una costante
- Esempi
 - `#define NMMAZ 10`
 - `#define VMAX 150.0`
 - `#define FALSE 0`
 - `#define TRUE 1`
- In questo caso l'andare a capo indica la fine dell'istruzione
- Il nome della costante viene anche detta **macro**



- In fase di compilazione ogni riferimento a macro viene sostituito con il valore corrispondente
- Alla fine di questa sostituzione la costante è come se non fosse stata dichiarata
- Per questo motivo il compilatore non riserva alcuna zona di memoria ad una macro

```
#define PI 3.14
```

```
int main() {  
    float area;  
    float raggio;  
    ...  
    area = raggio * raggio  
        * PI;  
    ...  
}
```



```
int main() {  
    float area;  
    float raggio;  
    ...  
    area = raggio * raggio  
        * 3.14;  
    ...  
}
```



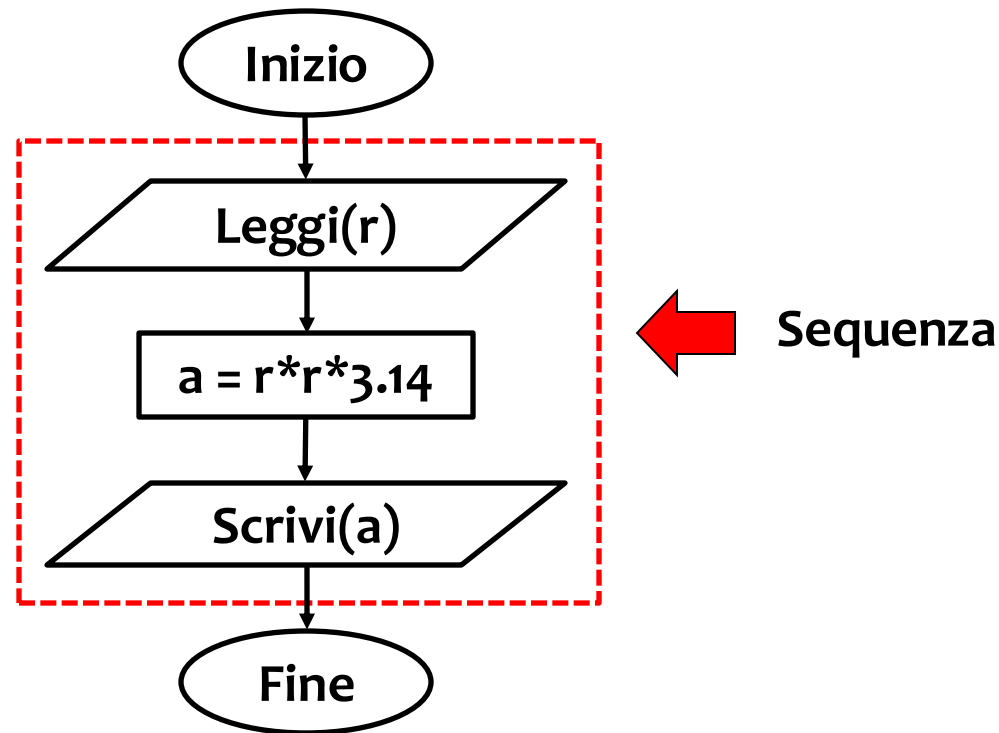
Contiene le istruzioni da eseguire:

- Istruzioni di assegnamento (=)
- Istruzioni di input/output
- Strutture di controllo:
 - Condizionali (if-else)
 - Iterative, o cicli (while, do-while e for)

Parte dichiarativa globale {
#include <stdio.h>
#define PI 3.14

Parte dichiarativa locale {
Parte esecutiva {
int main() {
float r, a;
scanf("%f", &r);
a = r * r * PI;
printf("%f", a);
return 0;
}
}

- Si disegni il diagramma di flusso di un algoritmo che chiede all'utente un lunghezza del raggio di un cerchio, calcola l'area del cerchio e visualizza il risultato





- Sequenze di istruzioni semplici
 - Ogni istruzione semplice è un'espressione che termina con ;
 - ; è detto il “terminatore” dell'istruzione
- Si possono raggruppare più istruzioni in sequenza tra { e } a costituire un blocco
 - Il blocco costituisce una “super-istruzione”
- Non è necessario il ; dopo } in quanto il blocco è già una istruzione
- Useremo i blocchi nelle istruzioni condizionali e di ciclo



Nelle istruzioni è possibile utilizzare i seguenti operatori:

- Aritmetici

- + (somma)
- - (sottrazione)
- * (prodotto)
- / (divisione)
- % (resto)

NOTA: Tra valori di tipo `int` e `char` viene effettuata la divisione intera. L'operatore `%` calcola il resto della divisione intera

- Assegnamento

- =

- Esistono anche operatori di confronto e logici (prossime lezioni)

char, int, float e double sono anche detti tipi aritmetici



- È un operatore e si indica con il simbolo = (uguale)
- A sinistra dell'operatore deve sempre esserci una variabile
- A destra può esserci
 - una costante
 - una variabile
 - un'espressione
- Esempi di istruzioni di assegnamento:

```
x = 23;    w = 'a';    w = a;    alfa = x + y;
```

- **Attenzione!** le variabili devono essere inizializzate
 - Bisogna sempre assegnare un valore ad una variabile prima di leggerne il contenuto in un'altra istruzione



1. Valutazione dell'espressione
 - Il valore delle variabili che vi compaiono si trova memorizzato nelle celle corrispondenti, e da lì è letto
2. Memorizzazione del risultato dell'espressione nella variabile a sinistra del simbolo =

Operatori C	Operazioni	Precedenza
()	Parentesi	Valutate per prime. Se ci sono degli annidamenti, si valuta prima la coppia più interna. Se ci sono più coppie allo stesso livello, si valuta da sinistra a destra.
* , / , %	Moltiplicazione, Divisione, Modulo	Valutate per seconde. Se ce ne sono diverse, si valutano da sinistra a destra.
+ , -	Addizione, Sottrazione	Valutate per terze. Se ce ne sono diverse, si valutano da sinistra a destra.
=	Assegnamento	Valutate per ultime. Se ce ne sono diverse, <u>si valutano da destra a sinistra</u> .



- In C, l'ingresso e l'uscita avviene tramite chiamate a due “istruzioni” particolari (in realtà sono due sottoprogrammi) disponibili nella libreria `stdio.h`
 - `printf()`
 - `scanf()`
- Per poterle utilizzare è necessario aggiungere la direttiva `#include <stdio.h>`



`printf(stringa di controllo, elementi da stampare);`

- `printf()` è l'identificatore riservato alla funzione
- stringa di controllo (racchiusa tra doppi apici `" "`), contiene:
 - caratteri alfanumerici da stampare a video
 - **caratteri di conversione** e/o di formato (preceduti dal simbolo `%`): utilizzati per interpretare i valori degli elementi da stampare
 - `%d` numero intero
 - `%f` numero in virgola mobile
 - `%c` carattere
 - **caratteri di controllo della stampa**: caratteri ASCII a cui non corrisponde alcun simbolo stampabile, hanno effetti di formato di stampa, come linea nuova (`\n`), tabulazione (`\t`), ecc.



FUNZIONAMENTO:

`printf (stringa di controllo, elementi da stampare) ;`

- Stampa i caratteri alfanumerici tra doppi apici e al posto dei caratteri di conversioni stampa il valore dell'identificatore corrispondente nella lista da stampare
- I caratteri di controllo stampa posizionano il cursore nella posizione opportuna



```
#include <stdio.h>

int main() {
    int num1;
    float num2;
    char car1;
    num1=1;
    num2=0.2;
    car1='r';
    printf("intero: %d\nrazionale: %f\ncarattere: %c\n",
           num1, num2, car1);
    return 0;
}
```

```
intero: 1
razionale: 0.2
carattere: r
```




`scanf(stringa di controllo, lista di variabili);`

- `scanf()` è l'identificatore riservato alla funzione
- stringa di controllo (racchiusa tra doppi apici ""), contiene:
 - **caratteri di conversione e/o di formato** (preceduti dal simbolo %): utilizzati per interpretare i valori degli elementi letti da tastiera, che devono essere memorizzati nelle variabili
 - %d numero intero
 - %f numero in virgola mobile
 - %c carattere
 - **variabili a cui associare il valore letto**: lista di identificatori di variabili. Le variabili devono essere precedute dal simbolo &. La lista è ordinata rispetto ai caratteri di conversione



FUNZIONAMENTO

`scanf(stringa controllo, lista di variabili);`

- Ad ogni pressione di un tasto, la funzione visualizza su video il carattere alfanumerico premuto
- La sequenza di tasti premuti deve terminare con la pressione del tasto **Invio**
- La funzione assegna alle variabili il valore binario del carattere letto

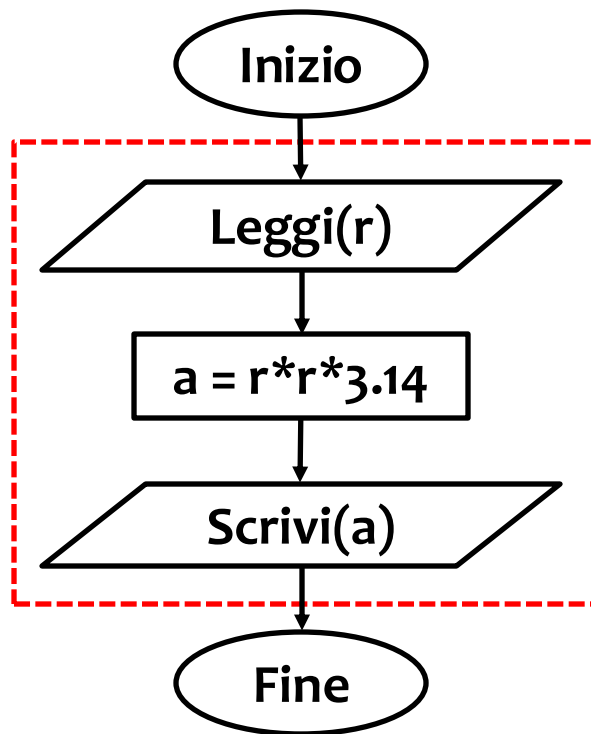


```
#include <stdio.h>

int main() {
    int num1;
    float num2;
    char car1;
    scanf("%d %f %c", &num1, &num2, &car1);
    printf("intero: %d\nrazionale: %f\ncarattere: %c\n",
        num1, num2, car1);
    return 0;
}
```

```
1 0.2 r
intero: 1
razionale: 0.2
carattere: r
```

- Si disegni il diagramma di flusso di un algoritmo che chiede all'utente un lunghezza del raggio di un cerchio, calcola l'area del cerchio e visualizza il risultato



```
#include <stdio.h>
#define PI 3.14
int main() {
    float r, a;
    scanf("%f", &r);
    a = r * r * PI;
    printf("%f", a);
    return 0;
}
```



- **Attenzione!** I tipi di dato delle variabili coinvolte in un'operazione devono essere identici (o almeno compatibili)
- Il compilatore controlla la compatibilità tra i tipi
- In alcune situazioni risolve la non compatibilità tramite delle regole di conversione automatica tra i tipi (**cast**)



- Operazioni di assegnamento
 - Il valore alla destra dell'operatore verrà convertito nel tipo della variabile alla sinistra
 - La conversione del tipo è «indolore» se il tipo di dato a destra è inferiore al tipo della variabile a sinistra dell'operatore, secondo la seguente gerarchia:

`char < int < float < double`

- Altrimenti non è esclusa perdita di informazione
 - Nell'assegnamento di un `int` ad una variabile `float` potrebbe esserci una perdita di informazione dovuta all'errore di approssimazione della rappresentazione `float` in numeri interi «molto grandi»
- Operazioni aritmetiche
 - Il valore di tipo inferiore viene convertito nel tipo superiore



- La conversione di un valore a `float` in un valore `int` comporta la perdita della parte frazionaria

```
int i;
```

```
float f;
```

```
f = 4.3; /*OK!*/
```

```
i = 4.3; /*assegnamento incongruente con  
perdita di informazione*/
```

```
f = 10; /*assegnamento incongruente senza  
perdita di informazione*/
```

```
i = 10; /*OK!*/
```



- Si può anche forzare la conversione di un valore da un tipo all'altro mediante l'operatore di cast ()
 - Si specifica tra parentesi tonde il tipo a cui convertire davanti al nome della variabile/costante

```
int a, b;
```

```
float f1, f2;
```

```
a = 5;
```

```
b = 2;
```

```
f1 = a / b;
```

```
f2 = (float) a / b;
```

CAST

La divisione viene effettuata tra numeri interi e poi il risultato è automaticamente convertito durante l'assegnamento

Il valore di a viene convertito a float prima della divisione; quindi la divisione sarà tra due valori float

- f1 contiene 2.0
- F2 contiene 2.5
- L'operazione di cast ha priorità maggiore rispetto a qualsiasi altra



```
#define D 5.0
int main(){
    float a;
    float b, c=0;
    int e;
    b = -11;
    a = D;           /* OK */
    D = a;           /* NO */
    a = 4 / 5;       /* Che cosa succede? Perché? */
    b = 4 / 5.0;     /* Che cosa succede? Perché? */
    b = 4 / 5 / 6.0; /* Che cosa succede? Perché? */
    b = 3 / (float)5 / 6; /* Che cosa succede? Perché? */
    e = (float)4 / 5; /* Che cosa succede? Perché? */
    ...
}
```



```
int main() {
    char a;
    char b, c = 'Q';
    a = '\n'; /* OK: i caratteri di escape sono
                caratteri a tutti gli effetti */
    c = 'ps'; /* NO: 'ps' non è una costante valida */
    a = '7';  /* Che cosa succede? */
    a = b;    /* Che cosa succede? */
    b = 7;    /* Che cosa succede? */
    a = 'g';
    a = a+1;  /* La var a contiene 'h' */
    a = 'a'+1; /* La var a contiene 'b' */
    a = 'g';
    a = a - 'a' + 'A'; /* La var a contiene 'G' */
    ...
}
```



- Il tipo `char` è rappresentato attraverso una codifica numerica intera (interi a 1 byte)
- **Attenzione!** `c = 1` è diverso da `c = '1'`
 - Nel primo caso a `c` è assegnato il valore 1
 - Nel secondo caso a `c` è assegnata la codifica del carattere 1 e quindi 49
- Supponendo che `c == '1'` con `c` di tipo `char`
 - `printf("%c", c);` stampa il carattere 1
 - `printf("%d", c);` è valido e stampa il numero 49
- Supponendo che `i == 90` con `i` di tipo `int`
 - `printf("%c", i);` è valido e stampa il carattere 'Z'
 - `printf("%d", i);` stampa il numero 90



- Quando scriviamo un programma possiamo commettere degli errori
 - Errori sintattici
 - Una o più istruzioni del programma non seguono le regole sintattiche del C
 - Il compilatore si accorge di tali errori, li segnala e non genera l'eseguibile
 - Errori semantici:
 - L'algoritmo non risolve il problema per cui è stato implementato
 - Il compilatore **NON** può accorgersi di tali errori



```
/* somma di due interi */  
#include <stdio.h>
```

```
int main()  
{  
    int integer1, integer2, sum;  
  
    printf("Enter first integer\n");  
    scanf("%d", &integer1 );  
    printf("Enter second integer\n")  
    scanf("%d", &integer2);  
    sum = integer1 + integer2;  
    printf("Sum is %d\n\n", sum;  
    return 0;  
}
```

«missing terminating " character».

Cioè mancano i doppi apici

«expected ';' before 'scanf'».

Cioè manca il ;

«expected ')' before ';' token».

Cioè manca la parentesi

ATTENZIONE: gli errori vanno corretti in ordine. Gli errori segnalati potrebbero celare al compilatore altri errori sintattici presenti nel programma; quindi bisogna ricompilare dopo aver corretto un po' di errori segnalati per verificare che non ne appaiano altri



```
/* somma di due interi */  
#include <stdio.h>
```

```
int main()  
{  
    int integer1, integer2, sum;  
  
    printf("Enter first integer\n");  
    scanf("%d", &integer1 );  
    printf("Enter second integer\n");  
    scanf("%d", &integer2);  
    sum = integer1 * integer2;  
    printf("Sum is %d\n\n", sum);  
    return 0;  
}
```

Ho scritto * al posto di +